DAILY TASK - 9

NAME : SANJEEV N
DATE : 18-08-2025

1. ADD SONGS

Edit Value | Schema


```
{
  "Id": 0,
  "Title": "Meesaya Murukku",
  "Artist": "Hip Hop Tamizha",
  "Album": "MM1",
  "Year": 2022
}
```


Response body


```
{
  "Id": 3,
  "Title": "Meesaya Murukku",
  "Artist": "Hip Hop Tamizha",
  "Album": "MM1",
  "Year": 2022
}
```

2. **LIST SONGS**


```
[
  {
    "Id": 1,
    "Title": "Perfect",
    "Artist": "Ed Sheeran",
    "Album": "Cold",
    "Year": 2020
  },
  {
    "Id": 2,
    "Title": "Bad Guy",
    "Artist": "Billie Eilish",
    "Album": "Rock",
    "Year": 2019
  },
  {
    "Id": 3,
    "Title": "Meesaya Murukku",
    "Artist": "Hip Hop Tamizha",
    "Album": "MM1",
    "Year": 2022
  }
]
```


3. **SEARCH BY ID**


```
{
  "Id": 1,
  "Title": "Perfect",
  "Artist": "Ed Sheeran",
  "Album": "Cold",
  "Year": 2020
}
```


CODE :

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import sessionmaker, Session, declarative_base
from fastapi import FastAPI, Depends, Request
from typing import List
import uvicorn
from pydantic import BaseModel

SQLALCHEMY_DATABASE_URL = "sqlite:///./songs.db"
```

```
engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)
Base = declarative_base()

class Song(Base):
    __tablename__ = 'songs'
    Id = Column(Integer, primary_key=True, index=True)
    Title = Column(String(100), nullable=False)
    Artist = Column(String(100), nullable=False)
    Album = Column(String(100), nullable=True)
    Year = Column(Integer, nullable=True)

Base.metadata.create_all(bind=engine)

app = FastAPI()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

class SongSchema(BaseModel):
    Id: int | None = None
    Title: str
    Artist: str
    Album: str | None = None
    Year: int | None = None

    class Config:
        orm_mode = True

@app.post("/songs/add", response_model=SongSchema)
def add_song(song: SongSchema, db: Session = Depends(get_db)):
    s = Song(
        Title=song.Title,
```

```
        Artist=song.Artist,
        Album=song.Album,
        Year=song.Year
    )
    db.add(s)
    db.commit()
    db.refresh(s)
    return s


@app.get("/songs", response_model=List[SongSchema])
def list_songs(db: Session = Depends(get_db)):
    all_songs = db.query(Song).all()
    return all_songs

@app.get("/songs/{song_id}", response_model=SongSchema)
def get_song(song_id: int, db: Session = Depends(get_db)):
    s = db.query(Song).filter(Song.Id == song_id).first()
    if not s:
        return {"error": "Song not found"}
    return s

@app.middleware("http")
async def addmiddleware(request: Request, call_next):
    print("Middleware Intercepted")
    response = await call_next(request)
    return response


if __name__ == "__main__":
    uvicorn.run("task:app", host="127.0.0.1", port=8000, reload=True)
```